

Lab 0 — Colab and Python

Runs with: Session 0 lecture · **Before:** Lab 1 · **Length:** two hours · **Bring:** a laptop and a Google account

This lab is not data science. It is getting Python and Colab working in your hands so that Lab 1 is spent on ideas rather than on setup. Nothing here assumes you have programmed before.

Starter notebook: open `Lab_0_Colab_and_Python_STARTER.ipynb` from the course page in Colab, and work through it alongside this sheet.

One rule: type every line yourself. Do not copy and paste, and do not just watch your neighbour. Typing is how the syntax gets into your hands, and copy-paste produces a room that has seen Python and cannot write it.

0:00–0:15 · Setup

Go to <https://colab.research.google.com> and sign in. Click **New notebook**.

In the first cell, type this and press Shift + Enter:

```
print("hello")
```

A Colab notebook is a page made of **cells**. A **code cell** holds Python and runs when you press Shift + Enter. A **text cell** holds writing — that is where you explain what you did and what it means.

The runtime is the computer running your code, in Google's data centre rather than on your laptop. It remembers everything you have run, which is why cells run out of order can produce answers that make no sense.

Do these three things now, before going further:

- **File → Save a copy in Drive.** Colab wipes temporary storage after about 90 minutes of inactivity. Anything not in Drive is gone, with no recovery.
- Rename your copy to `Lab0_yourname`.
- Find **Runtime → Restart and run all** in the menu. Do not click it yet. Just find it — you will use it in every lab this term.

0:15–0:35 · Task 1 — Values, variables, and types

A variable is a name for a value. Read `=` as **becomes**, not as **equals**. So `students = students + 1` is not nonsense: it means take what `students` is now, add one, and put it back.

```
students = 1200
course_name = "Fundamentals of Data Science"

print(students)
```

1.1 Make a variable `my_name` holding your name, and print it.

1.2 Predict each line before you run it, then run both:

```
print(14 + 1)
print("14" + "1")
```

The first gives 15. The second gives "141", because quotation marks make it text, and adding text joins it end to end. This is not a curiosity. A column of numbers read as text is the most common real bug in a first data project, and every calculation on it will be wrong or will raise.

1.3 Run `type()` on a number and on text. Then write one sentence in a text cell: why does it matter if a student ID is stored as a number rather than as text?

0:35–0:55 • Task 2 — Lists, and questions that return True or False

```
scores = [72, 65, 58, 91]

print(len(scores))
print(scores[0])
print(sum(scores) / len(scores))
```

Counting starts at zero. `scores[0]` is the first item; `scores[4]` does not exist and will raise `IndexError`. Everyone finds this annoying for about a week and then never thinks about it again.

2.1 Print the last item of `scores` in two different ways.

2.2 Say the answer out loud before you run this:

```
score = 72

print(score > 50)
print(score == 100)
print(score > 50 and score > 100)
```

One equals sign sets a value. Two equals signs ask a question. Writing `=` where you meant `==` is the most common beginner error in any language.

2.3 Write an `if` / `elif` / `else` that prints `Distinction` for 70 and above, `Pass` for 50 to 69, and `Fail` below 50. Test it with 45, 50 and 72.

Then answer in a text cell: does a score of exactly 50 pass, and where in your code is that decided? Keep that answer — it is the same question as “what does `passed` mean in the data dictionary?”, which you will meet in Week 3.

0:55–1:10 • Break

1:10–1:25 • Task 3 — Functions

Do not skip this task. The Week 1 five-minute diagnostic asks you to write a function. A function takes something in and gives something back.

```
def average(numbers):
    return sum(numbers) / len(numbers)

print(average(scores))
```

`def average(...)` names it. `numbers` is the input. `return` hands a value back.

3.1 Write `largest(numbers)` that returns the biggest value in a list. Test it on `scores`.

3.2 Run this and explain what happened:

```
def shows_average(numbers):
    print(sum(numbers) / len(numbers))

x = shows_average(scores)
print("x holds:", x)
```

A function that **prints** looks like it works, right up until you try to use the answer. A function with no `return` gives you `None`, which is the commonest cause of “why is my answer empty?”

1:25–1:40 • Task 4 — Break it on purpose

An error means Python stopped **before** doing something wrong. That is the good outcome. The dangerous case is code that runs and gives you a wrong answer — which is what Week 1 is about.

Read the last line first. It names the problem. The lines above it tell you where.

```
print(studnets)

NameError: name 'studnets' is not defined
```

4.1 Produce each of these errors deliberately, one per cell. Write down the last line of each message in a text cell underneath.

Error	How to cause it on purpose
NameError	Use a variable name you never created — misspell one.
TypeError	Add text to a number: "5" + 5
IndentationError	Put a line under a colon without indenting it.
IndexError	Ask for scores[10] when the list has four items.
FileNotFoundError	Read a CSV from a path that does not exist.

This is the most useful fifteen minutes of the session. A student who can read an error message unblocks themselves for the rest of the term. A student who cannot wait for the instructor, and in a lab of forty that means waiting twenty minutes.

1:40–1:50 • Task 5 — Your first real table

Now connect Drive and load actual data. This is the setup cell you will use in every lab for the rest of the course — it never changes.

```
from google.colab import drive
drive.mount('/content/drive')

DATA = "/content/drive/MyDrive/fds/data/" # change if your folder differs

import pandas as pd

students = pd.read_csv(DATA + "students.csv")
print(students.shape)
```

A permission window will appear asking to connect Drive. Accept it. This is the step that catches people — the window sometimes opens **behind** your browser.

You should see (1200, 7) — 1,200 rows and 7 columns. Printing .shape is the first thing to do with any file you open, every time. If the number of rows is not what you expected, stop and find out why.

5.1 Using students, print each of these:

- how many students are in each programme — use .value_counts()
- how many **distinct** values student_id has — use .nunique()
- the mean entry_score — use .mean()

5.2 Load `enrolments.csv` into a variable called `enrolments` and print its shape. Then answer in words, without running anything else: **is the number of rows in `enrolments` the number of students?** Why, or why not?

Do not look this up. Your answer will be either confirmed or corrected in the Week 1 lecture, and being wrong now costs nothing.

1:50–2:00 • Checkpoint and write-up

You are done when all of these are true:

- ☐ `print("hello")` ran
- ☐ you saved a copy to Drive and renamed it
- ☐ `largest()` works and returns a value
- ☐ you produced four different errors on purpose and wrote down the last line of each
- ☐ `students.shape` printed `(1200, 7)`
- ☐ Runtime → Restart and run all ran the whole notebook without unexpected errors

Write-up — about 100 words, in a text cell at the bottom of your notebook.

- Which error did you find hardest to understand, and what did the message turn out to mean?
- `enrolments.csv` has more rows than `students.csv`. What do you think one row of it is?

Submitting

1. **Runtime → Restart and run all.** Every cell must run top to bottom without errors — except the four in Task 4, which are meant to fail. Leave those in.
2. **File → Save a copy in Drive.**
3. Submit the notebook in the LMS. This lab is marked pass / revise / fail like every other lab.

Troubleshooting

Problem	Fix
Drive mount does nothing	The permission window is behind your browser. Look for it.
<code>FileNotFoundError</code> on a CSV	Your Drive path differs. Check <code>MyDrive/fds/data/</code> exactly, including capitals.
Shape is not <code>(1200, 7)</code>	You uploaded the wrong file, or an edited copy. Download the CSVs again.
Everything worked, now nothing does	The runtime disconnected. Runtime → Restart and run all.
My edit changed nothing	You edited the cell but did not run it. Shift + Enter.
A variable exists that I deleted	The runtime still remembers it. Restart and run all to be sure.
<code>ModuleNotFoundError</code>	Rare in Colab. Try <code>!pip install packagename</code> .

If something did not work today, tell your instructor before you leave. It will not fix itself, and Lab 1 assumes it works.